

后端架构导读

目的：赛场上要改一个行为时，能 30 秒内定位到「改哪个文件、改哪个配置」。详细设计仍看各文件头部注释，这里只做地图。

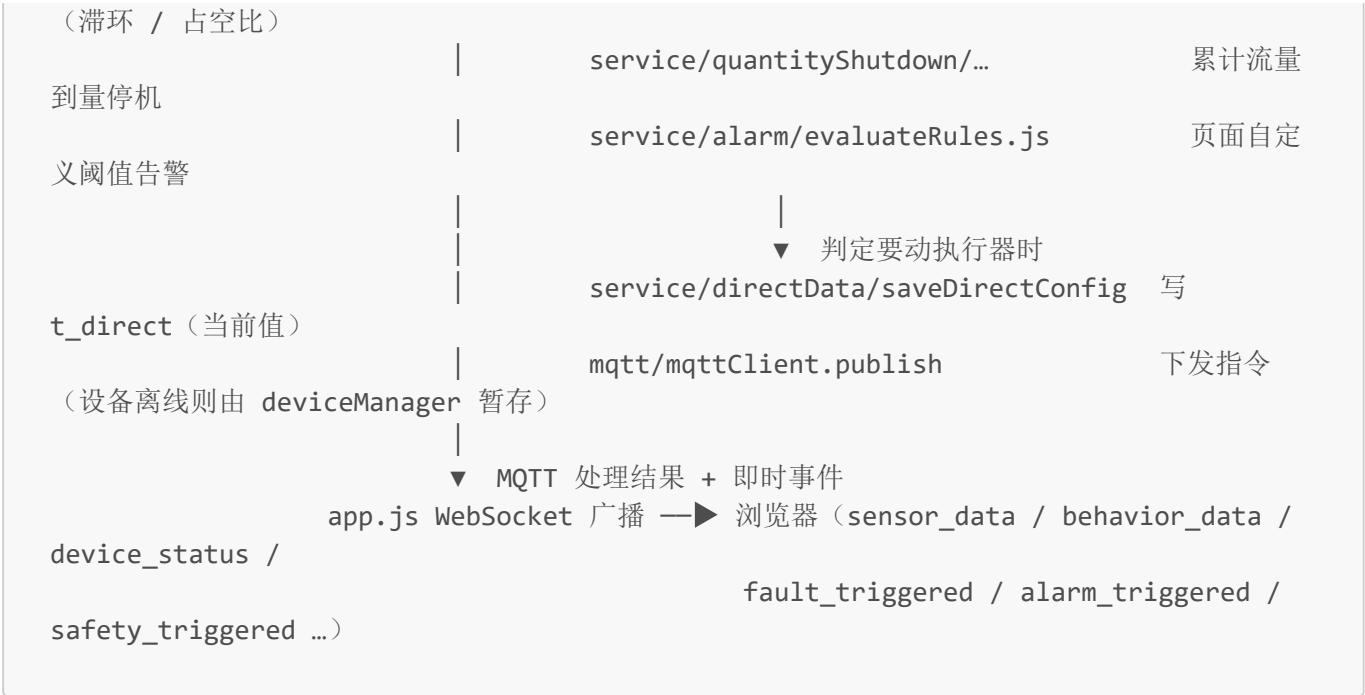
1. 一句话总览

Node + Express 单进程。三条链路跑在一起：

- 1. **HTTP API**：前端查数据 / 管设备 / 改配置 → `routes/sensorRoutes.js` 分发到 `controllers/` → `service/` → MySQL。
- 2. **MQTT 入库 + 自动控制**：设备上报 → `mqtt/` 解析入库 → 喂给一串**互相独立**的判定服务（安全连锁 / 故障检测 / 联动规则 / PID 恒温 / 恒流速 / 定量停机 / 告警规则）→ 需要动执行器时通过 MQTT 下发指令。
- 3. **WebSocket 推送**：`app.js` 把 MQTT 处理结果按节流间隔广播给前端；故障 / 告警 / 安全连锁这类即时事件不走节流，立即推。

2. 数据流向图





3. 目录职责表

目录	职责	典型文件
app.js	进程入口：Express 装配、CORS、静态托管、WebSocket、节流广播、启动初始化	—
routes/	唯一的 URL → controller 映射表	sensorRoutes.js
controllers/	只做「解析 req → 调 service → 返回 JSON」，不写业务逻辑（个别历史遗留除外）	device/、sensor/、system/
service/	业务逻辑与 SQL。按业务域分子目录	见下「按域」
mqtt/	MQTT 连接、订阅、路由、设备在线状态、离线指令暂存	mqttClient.js、messageRouter.js、deviceManager.js、index.js
config/	dbPool.js 连接池、env.js 读 .env、systemConfig.js 配置中心	—
utils/	与业务无关的纯工具：时间格式化、协议字段解析、设备号解析、近因过滤	helper.js、protocol.js、mappedData.js、recencyFilter.js
init/	一次性初始化脚本 / 建表 SQL	init_*.sql、init_pump_velocity_control.js
data/	运行期生成的持久化文件（配置、离线指令），已 gitignore	system-config.json、pending-commands.json

service/ 按域：sensor(查询) · tableData(分页表格统一数据源) · deviceData(设备增删改查) · directData(指令配置树 / 当前值) · computedMetrics / derivedMetric / cumulative / timeWindow / switchDuration(各

类图表指标) · `pidHeating` / `pumpVelocityControl` / `linkageRules` / `safety` / `faultStatus` / `quantityShutdown`(控制与保护) · `alarm`(阈值告警) · `operationHistory` / `errorHistory`(审计与异常记录) · `controlShared`(控制模块共用的小工具：换算、冷却、事件落库)

4. 「我想改 X」速查

想改的东西	去哪
新增 / 修改一个 HTTP 接口	<code>routes/sensorRoutes.js</code> + 对应 <code>controllers/</code>
设备上报的 JSON 字段名变了	不改代码 ：改数据库 <code>t_sensor_field_mapper</code> / <code>t_behavior_field_mapper</code> 的 <code>p_name</code> (<code>utils/mappedData.js</code> 按它对号入座到 <code>field1..field10</code>)
MQTT 地址 / 主题 / 心跳方式	配置中心 <code>MQTT_URL</code> / <code>MQTT_TOPICS</code> / <code>HEARTBEAT_MODE</code> (见 <code>CONFIG_CHEATSHEET.md</code>)，保存即热更新，无需重启
故障判定阈值 (压力/流量/温差...)	前端「指令配置」页对应指令项 (<code>t_direct</code> ，按 <code>prefix</code> 实时读)；删掉指令项才退回配置中心兜底
五种硬故障的判定逻辑本身	<code>service/faultStatus/faultStatus.js</code> (文件头有编号对照表)
正常工况联动规则的开关组合	配置中心 <code>LINKAGE_RULES</code> 逐条布尔开关；规则实现在 <code>service/linkageRules/linkageRules.js</code> 下半部分
PID 系数 $K_p/K_i/K_d$ 、控制周期、占空比上下限	「指令配置」页 PID 子项； <code>service/pidHeating/pidHeating.js</code> 只在指令项缺失时用 <code>PID_HEATING</code> 兜底
智能判定对接的现场接口形态 (同步/异步、JSON/表单/文本)	不改代码 ：配置中心 <code>INTELLIGENT_JUDGMENT</code> 各子键；适配逻辑在 <code>controllers/intelligent/recognize.js</code>
页面术语 / 分页大小 / 实时刷新间隔	配置中心 (<code>DEVICE_LABEL</code> 、 <code>DEFAULT_PAGE_SIZE</code> 、 <code>REALTIME_REFRESH_INTERVAL</code> ...)
部署参数 (端口、监听地址、CORS、API_TOKEN、NODE_ENV)	<code>.env</code> (模板见 <code>.env.example</code>)，改后需重启

5. 两条主链路时序

HTTP 查询 (以「传感器汇总数据」为例)

```

GET /api/dataByType?type=sensor&... → routes → controllers/sensor/tableData.js
→ service/tableData/getTableData.js
  └─ 读字段映射表（列名 → 中文展示名）
  └─ 拼 SQL（含派生指标 / 累计 / 时间窗口的内联子查询）
  └─ query MySQL
  └─ helper.js 拼单位、套 value_map 文案
→ { success, data: { list, total, fieldUnits, chartSettings } }

```

MQTT 入库 + 控制

```

设备发一条上报 → mqttClient 'message' → mqtt/index.js
  └─ 心跳判定 (HEARTBEAT_MODE: receive 用数据当心跳 / topic 用专门主题) →
deviceManager.onHeartbeat
  └─ messageRouter.route(topic) → 对应 handler → repository 落库
  └─ emit 'processedMessage' → app.js 节流广播

同一条数据并行喂给判定服务（各自 try/catch，一个抛错不影响其它）：
  safetyInterlock / faultStatus / linkageRules / pidHeating / pumpVelocityControl
/
  quantityShutdown / alarm → 命中就 saveDirectData + mqttClient.publish 下发

```

6. 三条贯穿全局的约定

1. **指令中心优先、配置中心兜底**：所有「现场要能调」的阈值/开关，先查 `t_direct`（按 `prefix`），查不到才用 `systemConfig` 的默认值。删掉指令项 = 关掉该项，程序不报错。
2. **设备编号两套**：MQTT 上报里的编号对应 `t_device.number`；系统内部一律用 `t_device.d_no`（没填就用 `number` 顶替）。`number` → `d_no` 的转换只在入库和心跳时做一次。
3. **field1..field10 槽位**：数据表没有语义列名，靠 `t_*_field_mapper` 把设备字段映射到槽位。看懂某个 `fieldN` 是什么，去查映射表，别在代码里找。